

CLEAN CODE - COMPLETE NOTES (ADVANCED VERSION)

"Clean code always looks like it was written by someone who cares." – Robert C. Martin

"Any fool can write code that a computer understands. Good programmers write code that humans understand."

1. What is Clean Code?

Clean code is code that is easy to read, understand, maintain, and extend. It reduces cognitive load and prevents bugs.

2. Naming: Bad vs Good

```
# BAD
def d(a, b):
    return a * b

# GOOD
def calculate_rectangle_area(width, height):
    return width * height
```

3. BAD CODE (REALISTIC EXAMPLE)

```
class Order:
    def __init__(self, t, p):
        self.t = t
        self.p = p

def calculate(o):
    if o.t == 1:
        if o.p > 100:
            return o.p * 0.9
        else:
            return o.p
    elif o.t == 2:
        if o.p > 200:
            return o.p * 0.8
        else:
            return o.p
    elif o.t == 3:
        if o.p > 300:
            return o.p * 0.7
        else:
            return o.p
    else:
        return o.p
```

4. GOOD CODE (CLEAN ARCHITECTURE)

```
class Order:
    def __init__(self, order_type, price):
        self.order_type = order_type
        self.price = price

class DiscountPolicy:
    def calculate(self, price):
        return price

class RegularDiscount(DiscountPolicy):
```

```

def calculate(self, price):
    return price * 0.9 if price > 100 else price

class PremiumDiscount(DiscountPolicy):
    def calculate(self, price):
        return price * 0.8 if price > 200 else price

class VIPDiscount(DiscountPolicy):
    def calculate(self, price):
        return price * 0.7 if price > 300 else price

class DiscountFactory:
    @staticmethod
    def create(order_type):
        if order_type == "regular":
            return RegularDiscount()
        elif order_type == "premium":
            return PremiumDiscount()
        elif order_type == "vip":
            return VIPDiscount()
        return DiscountPolicy()

def calculate_final_price(order):
    discount = DiscountFactory.create(order.order_type)
    return discount.calculate(order.price)

```

5. Refactoring SerialDate (Clean Code Insight)

SerialDate is a Java class designed to represent dates without time. Although it was already considered good code, it still contained: - unclear naming (SerialDate → better: DayDate) - unused methods - poor test coverage (~50%) - duplicated logic - excessive static methods Key improvements during refactoring: - Improve naming clarity - Replace constants with enums - Remove unused code - Increase test coverage (~90%) - Use Factory Pattern instead of direct instantiation - Reduce complexity and duplication

6. Core Principles

- Single Responsibility Principle (SRP) - Open/Closed Principle - Small Functions - No Magic Numbers - Replace Conditionals with Polymorphism - Write Tests First (TDD mindset)

7. Final Thought

"Leave the code better than you found it." – The Boy Scout Rule